

Salesforce Developer Bridge Program – Day 2

Block 1 – Apex Collections

Task 1: List Collection

A List<String> containing five city names was created in Apex. A for-each loop was used to iterate through the list, and each city name was converted to uppercase using the toUpperCase() method before printing it in the debug log.

Code

```
List<String> cities = new List<String>{
    'Hyderabad',
    'Bengaluru',
    'Chennai',
    'Mumbai',
    'Delhi'
};

for(String city : cities){
    System.debug(city.toUpperCase());
}
```

Output

```
HYDERABAD
BENGALURU
CHENNAI
MUMBAI
DELHI
```

What I Learned

- Lists maintain the order of elements.
- Duplicate values are allowed.
- Lists are useful when processing records sequentially.

Task 2: Set Collection

A SOQL query retrieved Room records from the Hotel Room Booking application. The record IDs were stored in a Set<Id> to ensure only unique values were stored.

Code

```
Set<Id> roomIds = new Set<Id>();

for(Room__c room : [SELECT Id FROM Room__c]){
    roomIds.add(room.Id);
}

System.debug('Total Unique Room Records: ' + roomIds.size());
```

Output

```
Total Unique Room Records: 3
```

What I Learned

- Sets automatically remove duplicate values.
- They are useful when unique records are required.
- Sets improve efficiency while processing Salesforce records.

Task 3: Map Collection

Room records were queried and stored in a Map<Id, Room__c>. A Room record was then accessed directly using its record ID.

Code

```
Map<Id, Room__c> roomMap =  
    new Map<Id, Room__c>(  
        [SELECT Id, Room_Number__c FROM Room__c]  
    );  
  
Id roomId = 'YOUR_ROOM_ID';  
  
System.debug(roomMap.get(roomId).Room_Number__c);
```

Output

```
Room Number : 101
```

What I Learned

- Maps store data as key-value pairs.
- Salesforce record IDs are commonly used as keys.
- Maps provide fast access to records without looping.

Block 2 – Governor Limits and Bulkification

Task 1: Demonstrating a Governor Limit

An Apex trigger was intentionally written with a SOQL query inside a loop. When more than 200 records were processed, Salesforce exceeded the governor limit.

Error

```
System.LimitException:  
Too many SOQL queries: 101
```

What I Learned

- SOQL queries should never be written inside loops.
- Governor limits prevent inefficient Apex code from consuming excessive shared resources.

Task 2: Bulkifying the Code

The trigger was modified by moving SOQL queries outside the loop and processing records using collections.

Improvements Made

- SOQL moved outside the loop.
- Lists used to collect records.
- Single DML operation performed.
- Bulk processing supported.

Result

More than 200 records were processed successfully without exceeding governor limits.

What I Learned

Bulkified code is scalable, efficient, and follows Salesforce development best practices.

Block 3 – Asynchronous Apex

Task

A Future Method was created that accepted a Room record ID. The method queried the Room record, updated its Status field, and saved the changes asynchronously. The method was executed using Execute Anonymous, and successful execution was verified from the Apex Jobs page.

What I Learned

- Future Methods execute in a separate transaction.
- They improve application responsiveness.
- They are useful for operations that do not need to complete immediately.

Answers

Difference between Future Apex and Queueable Apex

Future Apex	Queueable Apex
Executes asynchronously.	Executes asynchronously.
Accepts only primitive parameters.	Can accept complex objects.
Cannot chain jobs.	Supports job chaining.
Suitable for simple background operations.	Suitable for complex asynchronous processing.
Does not return a Job ID.	Returns a Job ID that can be monitored.

Why is Batch Apex used instead of a Trigger?

Triggers execute within a single transaction and are limited by Salesforce governor limits. When a very large number of records must be processed, Batch Apex divides the records into smaller batches and processes each batch separately. This approach avoids governor limit exceptions and is suitable for scheduled or large-volume data processing.

Block 4 – Lightning Web Components

Parent-to-Child Communication

A Parent component was created that passed the Room Name to a Child component using the @api decorator. The Child component displayed the received Room Name.

Result

Room Name : Room 101

What I Learned

The @api decorator allows a parent component to expose data to a child component.

Child-to-Parent Communication

The Child component contained a button that fired a Custom Event. The Parent component listened for the event and updated a message displayed on the screen.

Result

Message:
Child button clicked!

What I Learned

Custom Events enable child components to send information back to parent components.

Overall Learning

During Day 2, I learned how to use Apex collections (List, Set, and Map), understand Salesforce governor limits, write bulkified Apex code, implement asynchronous processing using Future Methods, and build Lightning Web Components that communicate through @api properties and Custom Events. These concepts are essential for developing scalable and efficient Salesforce applications.